

🌟 RS003 Week 12 — English Worksheet

Topic: A Falling Starfield and Dropping Meteors · **Course:** Pygame Space Shooter · **Time:** about 45 minutes

This week is about **thinking about** and **explaining** code that makes a background move. Stars fall down the screen, wrap back to the top when they leave the bottom, and meteors drift down at different speeds. Each star and meteor is a small **dictionary** of facts, and the program loops over a **list** of them every frame.

💬 Words to know: **list**, **dictionary**, **fall**, **wrap-around**, **speed**

1 · Predict 🌟

Read each piece of code and write what you think it does. You do not run anything — just think and write.

```
stars = []
for i in range(80):
    stars.append({"x": random.randint(0, WIDTH), "y": random.randint(0, HEIGHT),
                  "speed": random.randint(1, 4)})
```

This builds a list of 80 stars. What three facts does each star remember?


```
star["y"] = star["y"] + star["speed"]
if star["y"] > HEIGHT:
    star["y"] = 0
```

Each star moves down a little every frame. What happens when a star goes off the bottom edge?


```
meteor["y"] = meteor["y"] + meteor["speed"]
pygame.draw.circle(screen, (120, 120, 140), (meteor["x"], int(meteor["y"])),
meteor["size"])
```

Each meteor falls down at its own speed and each has a different **size**. What do the meteors look like together?


```
meteors = []
for i in range(5):
    meteors.append({"x": random.randint(0, WIDTH), "y": random.randint(-200, 0),
"speed": random.uniform(1.0, 3.0), "size": random.randint(15, 30)})
```

This builds a list of 5 meteors. Why does each meteor start with a **y** value that can be a negative number?

2 · Spot the Bug 🐛

Each block below was meant to do something but is broken. Write the fix, then explain why the original was wrong.

Bug A — All the stars are supposed to fall at different speeds, but instead they all fall together in a solid wall.

```
stars.append({"x": random.randint(0, WIDTH), "y": random.randint(0, HEIGHT),  
"speed": 2})
```

Hint: every star has the same `speed`, so they all move the same.

Write the fixed code:

Why was it wrong? Why does your fix work?

Bug B — Stars should keep falling forever, but once they reach the bottom they never come back and the screen goes empty.

```
star["y"] = star["y"] + star["speed"]  
pygame.draw.circle(screen, (255, 255, 255), (star["x"], star["y"]), 2)
```

Hint: there is no wrap-around to send the star back to the top.

Write the fixed code:

Why was it wrong? Why does your fix work?

Bug C — The program crashes when drawing a meteor because a position has a decimal point.

```
meteor["y"] = meteor["y"] + meteor["speed"]
pygame.draw.circle(screen, (120, 120, 140), (meteor["x"], meteor["y"]),
meteor["size"])
```

Hint: `speed` is a decimal, so `meteor["y"]` becomes a decimal. Positions must be whole numbers.

Write the fixed code:

Why was it wrong? Why does your fix work?

3 · Explain the Code

Read this working example carefully. Then answer the questions below it. You are only reading — do not run it.

```
import pygame
import random
pygame.init()

WIDTH, HEIGHT = 800, 600
screen = pygame.display.set_mode((WIDTH, HEIGHT))
clock = pygame.time.Clock()

stars = []
for i in range(80):
    stars.append({"x": random.randint(0, WIDTH), "y": random.randint(0, HEIGHT),
                  "speed": random.randint(1, 4)})

meteors = []
for i in range(5):
    meteors.append({"x": random.randint(0, WIDTH), "y": random.randint(-200, 0),
                    "speed": random.uniform(1.0, 3.0), "size": random.randint(15, 30)})

running = True
while running:
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            running = False

    screen.fill((10, 12, 40))

    for star in stars:
        star["y"] = star["y"] + star["speed"]
        if star["y"] > HEIGHT:
            star["y"] = 0
            star["x"] = random.randint(0, WIDTH)
        pygame.draw.circle(screen, (255, 255, 255), (star["x"], star["y"]), 2)

    for meteor in meteors:
        meteor["y"] = meteor["y"] + meteor["speed"]
        if meteor["y"] > HEIGHT + meteor["size"]:
            meteor["y"] = -meteor["size"]
            meteor["x"] = random.randint(0, WIDTH)
        pygame.draw.circle(screen, (120, 120, 140), (meteor["x"],
int(meteor["y"])), meteor["size"])

    pygame.display.flip()
    clock.tick(60)
```

```
pygame.quit()
```

What does the line `for star in stars:` do — what is it looping over?

The line `star["y"] = star["y"] + star["speed"]` runs every frame. What does it make the star do?

Look at the wrap-around for the stars. How does the code send a star back to the top, and where does the star reappear?

Why does the meteor wrap-around use `> HEIGHT + meteor["size"]` instead of just `> HEIGHT` ?

Why is `int(meteor["y"])` used when drawing the meteor but not when drawing the star?

4 · Explain Your Lesson Code 🎥

In today's live lesson you wrote your own moving background. Now make a short video on your phone explaining the code **you** wrote. You may show it running. Try to use these words: **list**, **dictionary**, **fall**, **wrap-around**, **speed**.

1. Show your moving background and point at your screen.
2. Read the line that builds one star or meteor out loud and name its facts.
3. Point at your wrap-around code and explain what it does.
4. Explain one part of your code that was tricky and how you got it working.

Write what you will say in your video. Plan it here before you record.

Submit 

Send this worksheet + a video explaining your lesson code to teacher on KakaoTalk.